Mid - C++ Merge Sort Code

write me cpp code 1d peak finding using recursive divide and conq

Certainly! Here's a simple C++ implementation of **1D peak finding** using the recursive **Divide and Conquer** approach:

A **peak element** in an array is one that is **not smaller than its neighbors**. The divide and conquer method checks the middle, and recursively searches the half that might contain a peak.

---

cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Recursive function to find a peak element
int findPeakUtil(const vector<int>& arr, int low, int high, int n) {
    int mid = low + (high - low) / 2;

    // Check if mid is a peak
    if ((mid == 0 || arr[mid-1] <= arr[mid]) &&
        (mid == n-1 || arr[mid+1] <= arr[mid])) {
        return mid;
    }
    // If left neighbor is greater, then peak must be in left half
    else if (mid > 0 && arr[mid-1] > arr[mid]) {
        return findPeakUtil(arr, low, mid-1, n);
    }
    // Else peak must be in right half
    else {
        return findPeakUtil(arr, mid+1, high, n);
    }
}
```

```
int findPeak(const vector<int>& arr) {
    int n = arr.size();
    return findPeakUtil(arr, 0, n-1, n);
}

int main() {
    vector<int> arr = {1, 3, 20, 4, 1, 0};
    int peakIdx = findPeak(arr);
    cout << "Peak found at index: " << peakIdx << " (Value: " << arr[peakIdx] << ")" << endl;
    return 0;
}
```

How it works:

- The function recursively finds the middle of the current segment.
- If the mid is a peak, returns its index.
- Otherwise, it searches left or right based on which neighbor is bigger.

You can change the input `arr` to test with different arrays.

Let me know if you want comments/explanation for each line!

write me pseudocode for it

Absolutely! Here is clear, easy-to-follow **pseudocode** for recursive 1D peak finding using divide and conquer:

Pseudocode: 1D Peak Finding (Recursive, Divide and Conquer)

```
sql

function findPeak(arr, low, high, n):
    mid = low + (high - low) // 2

    if (mid == 0 or arr[mid-1] <= arr[mid]) and (mid == n-1 or arr[mid+1] <= arr[mid]):
```

```
        return mid  // Peak found

    else if (mid > 0 and arr[mid-1] > arr[mid]):
        return findPeak(arr, low, mid-1, n)  // Search left half

    else:
        return findPeak(arr, mid+1, high, n) // Search right half


// To call the function:
function main():
    arr = [your array]
    n = length of arr
    peakIndex = findPeak(arr, 0, n-1, n)
    print "Peak found at index:", peakIndex, "Value:", arr[peakIndex]
```

Explanation:

- The function checks the middle element.
- If the middle is a peak, return its index.
- If the left neighbor is larger, recursively search the left half.
- Otherwise, recursively search the right half.

Let me know if you need this explained with a step-by-step example!

now write cpp code for 2d peak finding

Of course! Here's a simple C++ implementation for **2D Peak Finding** using the **Divide and Conquer** method. In 2D, a peak is an element that is **not smaller than its four neighbors** (up, down, left, right).

We'll use the method where we select the middle column, find the global maximum in that column, and compare it with its neighbors to decide which half to recurse on.

C++ Code: 2D Peak Finding (Recursive Divide and Conquer)

cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Function to find the index of the maximum element in a specific column
int findMaxInColumn(const vector<vector<int>>& matrix, int col) {
    int maxRow = 0;
    int n = matrix.size();
    for (int i = 1; i < n; ++i) {
        if (matrix[i][col] > matrix[maxRow][col])
            maxRow = i;
    }
    return maxRow;
}

// Recursive function to find a 2D peak
pair<int, int> findPeak2DUtil(const vector<vector<int>>& matrix, int left, int right) {
    int numRows = matrix.size();
    int numCols = matrix[0].size();
    int midCol = left + (right - left) / 2;

    // Find the maximum element in the middle column
    int maxRow = findMaxInColumn(matrix, midCol);

    // Check for peak in the middle column
    int current = matrix[maxRow][midCol];
    int leftNeighbor = (midCol - 1 >= 0) ? matrix[maxRow][midCol - 1] : INT_MIN;
    int rightNeighbor = (midCol + 1 < numCols) ? matrix[maxRow][midCol + 1] : INT_MIN;

    if (current >= leftNeighbor && current >= rightNeighbor) {
        return {maxRow, midCol}; // Peak found
    } else if (leftNeighbor > current) {
        // Recurse on the left half
        return findPeak2DUtil(matrix, left, midCol - 1);
    } else {
        // Recurse on the right half
        return findPeak2DUtil(matrix, midCol + 1, right);
```

```
    }
}

pair<int, int> findPeak2D(const vector<vector<int>>& matrix) {
    int numCols = matrix[0].size();
    return findPeak2DUtil(matrix, 0, numCols - 1);
}

int main() {
    vector<vector<int>> matrix = {
        {10, 8, 10, 10},
        {14, 13, 12, 11},
        {15, 9, 11, 21},
        {16, 17, 19, 20}
    };

    auto peak = findPeak2D(matrix);
    cout << "Peak found at: (" << peak.first << ", " << peak.second << ") with value: "
         << matrix[peak.first][peak.second] << endl;

    return 0;
}
```

How it works:

- The function always works with the current middle column.
- It finds the row with the largest value in that column.
- If the value is a peak (not smaller than left and right neighbors), it returns the position.
- Otherwise, it recurses into the side with the bigger neighbor.

Let me know if you need **step-by-step explanation** or want the **pseudocode** for this algorithm!

walk me through a proper example every step by satep to teach me the algorithm. show mw the matrix in every step

Absolutely! Let's walk through a full example of **2D Peak Finding (Divide and Conquer)** step by step, showing the matrix at each stage and all decisions.

Algorithm Recap

1. Pick the middle column.
2. Find the maximum element in that column.
3. If it's a peak (no neighbor is greater: left/right), stop!
4. If a neighbor is greater, recurse into that half (left/right).

Example Matrix

Let's use this 4×4 matrix:

```csharp
[10,  8, 10, 10]
[14, 13, 12, 11]
[15,  9, 11, 21]
[16, 17, 19, 20]
```

Step 1: Start With All Columns

`left = 0`, `right = 3`
Middle column: `(0 + 3) / 2 = 1` → **Column 1**

Matrix with middle column highlighted:

```scss
```

```
[10,  8, 10, 10]
[14, 13, 12, 11]
[15,  9, 11, 21]
[16, 17, 19, 20]
    ^
  (column 1)
```

Step 2: Find the Max in Middle Column (Column 1)

Column 1 values: 8, 13, 9, 17
Max value: 17 at **row 3**

So, current cell: **(3, 1)** = 17

Step 3: Compare to Left/Right Neighbors

- **Left neighbor:** (3, 0) = 16
- **Right neighbor:** (3, 2) = 19

Current value (17) < right neighbor (19)

- We need to **search right half** (columns 2 and 3).

Step 4: Recurse on Right Half

`left = 2`, `right = 3`
Middle column: `(2 + 3) / 2 = 2` → **Column 2**

Matrix:

```scss

[10,  8, 10, 10]
[14, 13, 12, 11]
[15,  9, 11, 21]
[16, 17, 19, 20]
```

```
        ^
(column 2)
```

Step 5: Find Max in Column 2

Column 2 values: 10, 12, 11, 19
Max value: 19 at **row 3**

So, current cell: **(3, 2)** = 19

Step 6: Compare to Neighbors

- **Left neighbor:** (3, 1) = 17
- **Right neighbor:** (3, 3) = 20

Current value (19) < right neighbor (20)

- We need to **search right half** (column 3).

Step 7: Recurse on Right Half

`left = 3`, `right = 3`
Only **Column 3** is left.

Matrix:

```
scss

[10,  8, 10, 10]
[14, 13, 12, 11]
[15,  9, 11, 21]
[16, 17, 19, 20]
          ^
        (column 3)
```

Step 8: Find Max in Column 3

Column 3 values: 10, 11, 21, 20

Max value: 21 at **row 2**

So, current cell: **(2, 3)** = 21

Step 9: Compare to Neighbors

- **Left neighbor:** (2, 2) = 11
- **No right neighbor** (column 3 is the last column)

Current value (21) > left neighbor (11)

- Since there is **no right neighbor** (edge of matrix), and it's bigger than the only neighbor,
- **This is a peak!**

Conclusion

Found Peak:

- **Row:** 2
- **Col:** 3
- **Value:** 21

Visual Recap of Steps

1. Start with column 1 → max at (3,1)=17, right neighbor (19) is bigger → go right
2. Column 2 → max at (3,2)=19, right neighbor (20) is bigger → go right
3. Column 3 → max at (2,3)=21, no right neighbor, left neighbor smaller → **peak found!**

Let me know if you want another example, want to see this with a different matrix, or want an animation/diagram to illustrate the search!

now write me pseodocode

Absolutely! Here is **clear pseudocode** for the 2D Peak Finding algorithm using recursive divide and conquer. I'll include comments to make each step understandable.

2D Peak Finding (Recursive Divide & Conquer) — Pseudocode

sql

```
function find2DPeak(matrix, left, right):
    nRows = number of rows in matrix
    midCol = left + (right - left) // 2

    // Step 1: Find the row index of the max element in midCol
    maxRow = 0
    for i = 1 to nRows-1:
        if matrix[i][midCol] > matrix[maxRow][midCol]:
            maxRow = i

    // Step 2: Check left and right neighbors
    current = matrix[maxRow][midCol]
    leftNeighbor  = if midCol > 0       then matrix[maxRow][midCol-1] else -infinity
    rightNeighbor = if midCol < numCols-1 then matrix[maxRow][midCol+1] else -infinity

    // Step 3: Is current a peak?
    if current >= leftNeighbor and current >= rightNeighbor:
        return (maxRow, midCol)   // Return the coordinates of the peak

    // Step 4: Otherwise, recurse left or right
    else if leftNeighbor > current:
        return find2DPeak(matrix, left, midCol-1)
    else:
        return find2DPeak(matrix, midCol+1, right)
```

```
// Initial call:
function main():
    matrix = [your 2D array]
    numCols = number of columns in matrix
    peakPosition = find2DPeak(matrix, 0, numCols-1)
    print "Peak found at:", peakPosition, "with value:", matrix[peakPosition.row][peakPosition.col]
```

Summary of the pseudocode steps:

- Pick the middle column.
- Find the global maximum in that column.
- If it’s a peak, return its position.
- If a neighbor is bigger, recurse into the corresponding half.

Let me know if you’d like a **visual diagram** or a **step-by-step pseudocode with an example matrix**!